Int8 Code Correlator and E1B/E1C CBOC Update

Date: 2026-07-08

Summary

The standard correlator `sdr_corr_std()` was extended to accept spreading codes with arbitrary int8 values instead of {-1, 0, +1} only, with no measurable throughput change on AVX2. Using this, the Galileo E1B/E1C code generation was changed from the BOC(1,1) approximation to the full CBOC(6,1,1/11) waveform, removing the 10*log10(10/11) = 0.41 dB correlation power loss of the approximation (realized with a wide-band front end; see Band-width consideration below).

Signals with -1/0/+1 codes produce bit-identical correlations before and after this change.

Changed files: `src/sdr_func.c`, `src/sdr_code.c`, `src/sdr_ch.c`, `src/pocket_sdr.h`, `test/src/test_sdr_func.c`, `test/src/test_sdr_perf.c`.

Problem

The AVX2 kernel of `dot_IQ_code_acc()` multiplied IF data by the code with `_mm256_sign_epi8`, which only realizes multiplication by -1/0/+1. Because of this, multi-level waveforms had to be approximated by 2-level codes:

- E1B/E1C: CBOC(6,1,1/11) replaced by BOC(1,1), losing the beta component (1/11 of the signal power, 0.41 dB of correlation amplitude 10*log10(10/11)).
- E5 AltBOC and B1C QMBOC are similarly approximated (not addressed by this update).

A previous attempt to lift the restriction by widening data and code to int16 (`unpack` + `madd_epi16`) roughly doubled the vector operation count and was rejected as too slow.

Method

Signed x signed int8 products through `maddubs`

The key observation is that the existing kernel already uses `_mm256_maddubs_epi16(a, b)` (unsigned u8 x signed s8 -> saturated s16 pair sum), but only as an I/Q lane extractor with a constant 0/1 pattern in `a`. The code amplitude can be moved into that instruction instead:

- `a` = IF data biased by +128, i.e. `(uint8_t)(IQ + 128)`, range [1, 255] (|IQ| <= 127 by the `SDR_CSCALE` design),
- `b` = int8 code, expanded so that Q (or I) byte lanes are zero: `cI = (c0,0,c1,0,...)` by one `shuffle_epi8`, `cQ = (0,c0,0,c1,...)` derived by `slli_epi16(cI, 8)`.

Since one lane of every byte pair is zero, the pair sum never exceeds 255 * 127 = 32385 < 32767, so `maddubs` saturation cannot occur for any int8 code value. The rest of the kernel (`madd_epi16` with ones, int32 accumulators) is unchanged. The operation count per 16 complex samples is the same as the previous kernel (the `sign_epi8` is replaced by the `slli`), which is why there is no slowdown.

Alternatives measured in a stand-alone microbenchmark (Ryzen AI 9 HX 370, gcc -O3 -mavx2, 24000-sample period, 4096-sample chunks, 6 correlators):

kernel	relative speed
previous (`sign_epi8`, -1/0/+1)	1.00
bias method (this update)	1.00
abs+sign method (no correction)	0.75
bias + (c,0)-expanded code bank	1.03 (2x bank memory)

Zero-cost biasing: `mix_tbl_b`

The +128 bias is not applied as a separate pass. A second carrier-mix LUT `mix_tbl_b` (the entries of `mix_tbl` XOR 0x80, AVX2 build only, 128 KB static) is generated in `sdr_func_init()`, and `sdr_corr_std()` mixes its chunks through that table, so the biased samples come out of the fused carrier mixing for free. An earlier version used an explicit XOR pass over each chunk and measured 3-5% slower.

`sdr_mix_carr()` and all other consumers keep using the unbiased `mix_tbl`; the biased representation never escapes `sdr_corr_std()`.

Exact bias removal by code prefix sums

The biased accumulation satisfies

```
sum((d + 128) * c) = sum(d * c) + 128 * sum(c)
```

so the bias term is removed exactly using prefix sums of each resampled code bank slice (`trk->code_sum`, int32, (N+1) x SDR_N_CODES, built in `trk_new()` under `#if defined(AVX2)` only). Because the accumulation ranges of the two wrap-around partial sums in `sdr_corr_std()` are [N-j, N) and [0, N-j) with j the code shift, the correction needs only `code_sum[N-j]` and `code_sum[N]` per correlator. The scalar tail of `dot_IQ_code_acc()` also accumulates biased values so a single full-range correction applies.

Memory cost: 40*(N+1) bytes per channel (about 1 MB at 24 Msps x 1 ms, 3.8 MB for E1 at 24 Msps x 4 ms) on AVX2 builds only; not allocated on NEON/scalar builds, which need no correction.

Overflow bound

The int32 accumulators bound the guaranteed-safe integration length by

```
N * 255 * max|code| < 2^31
```

With the CBOC scale below (max|code| = 40) this gives N <= 210k samples per call, covering E1 4 ms up to about 50 Msps. `sdr_corr_std()` documents the condition.

Correlation scale normalization

`sdr_corr_std()` takes the code amplitude scale (1 for -1/0/+1 codes) and divides the output correlations by it, so downstream processing (DLL/PLL, C/N0, nav decoding, logs) sees the same correlation scale as before.

NEON

- `vdotq_s32` path (dotprod): already computes full int8 x int8 products; no change other than comments. No bias, no correction.
- non-dotprod fallback: `vmlal_s8` accumulated into int16 lanes, which would overflow after a few samples with int8 codes; replaced by `vmull_s8` + `vpadalq_s16` into int32 lanes (drain-free).

E1B/E1C CBOC code generation

`gen_code_E1B()` / `gen_code_E1C()` now generate 12 sub-chips per primary chip via `mod_code_CBOC()`:

```
value = chip * round(SDR_CBOC_SCALE * (alpha * sc1 +/- beta * sc6))
```

with alpha = sqrt(10/11), beta = sqrt(1/11), `SDR_CBOC_SCALE` = 32, the sign `+` for E1B (data) and `-` for E1C (pilot) per the Galileo OS SIS ICD. The resulting levels are +/-40 and +/-21 (exact: 40.16, 20.86); the quantization loss is below 0.001 dB. The generated code length changes from 2 x 4092 to 12 x 4092; `sdr_code_len()` (chips) is unchanged.

The acquisition path (`sdr_gen_code_fft()`) consumes the multi-level chips as float and needs no change; acquisition C/N0 estimation is ratio-based and scale-invariant.

API changes

- `sdr_corr_std()`: added `const int32_t *code_sum` and `int scale` parameters.
- `sdr_code_scale()`: new; returns the code amplitude scale of a signal (`SDR_CBOC_SCALE` for E1B/E1C, else 1).
- `sdr_trk_t`: added `code_sum` and `code_scale` members.

- `SDR_CBOC_SCALE` (= 32) added to `pocket_sdr.h`.
- `sdr_corr_std_cpx()` (python support) scales the float code by `SDR_CBOC_SCALE` internally and builds the prefix sums on the fly; results for +/-1 codes are unchanged.

Verification

- Unit tests (17 binaries) all pass, including the reference-vs-SIMD equivalence test `test_sdr_corr_std_equiv()` extended with an int8 multi-level code case (tolerance 1e-9, exercising the bias correction).
- L1CA regression: acquisition results and tracking SIGNAL FOUND parameters are identical between the previous and the new build (as expected -- bias + exact correction reproduces the same integer sums).
- E1B/E1C on real data (E1E5B_20220114 sample, 24 Msps IQ, 30 s): three Galileo satellites tracked; E1B reaches frame sync and decodes 11-13 I/NAV pages with zero errors; E1C holds secondary code sync; no loss of lock.
- Throughput: interleaved A/B runs of `test_sdr_perf` (old/new alternating to cancel thermal drift) show identical `sdr_corr_std` throughput within +/-2%. Note: on laptops a single-sided comparison can show 10-20% apparent differences from thermal throttling alone.

Band-width consideration

The 0.41 dB CBOC gain assumes the BOC(6,1) component (+/-6.138 MHz) is present in the IF data, i.e. a front-end band width of roughly 14 MHz or more. With a narrow IF filter (e.g. 4.2 MHz), the beta component is filtered out and the CBOC replica correlates it against noise only, making the result about 0.41 dB *worse* than the BOC(1,1) replica (measured: about -0.1 dB in estimated C/N0, -4.6% in P amplitude on the 4.2 MHz E1E5B sample). Related: `THRES_LOST` in `sdr_ch.c` is an absolute amplitude threshold for secondary code sync, so the reduced P amplitude slightly increases the chance of a spurious resync on narrow-band data.

A possible follow-up is a receiver option to select the BOC(1,1) or CBOC replica per RF channel band width.

Remaining work

- E5 AltBOC (`dot_IQ_cpx_code()` / E5ABQ banks): the same bias method applies; the correction needs two prefix-sum banks, sum(cI + cQ) and sum(cI - cQ).
- B1C QMBOC(6,1,4/33) and I1SP CBOC: same approach as E1B/E1C if desired.
- `python/sdr_code.py` still generates the BOC(1,1) approximation for E1B/E1C, so the python tools do not match the C library for these signals.
- Note: the secondary code polarity (`sec_pol`) of E1C may differ from the previous build on the same data; the PLL Costas ambiguity settles arbitrarily and the polarity is resolved by the secondary code sync as designed.